

Neural Networks I: Foundations and Training

Lecture 9 – Introduction to Modern Machine Learning

Steven Campbell

Imperial College London Summer School

June 2026

Outline

1. Why neural networks?
2. Building blocks
3. Training neural networks
4. Generalisation and practice
5. Looking ahead to the problem class

Why neural networks?

Where this lecture fits

Earlier lectures introduced:

- supervised learning, loss functions, regularisation and cross-validation;
- k -nearest neighbours, linear models, decision trees and random forests;
- online learning as sequential prediction and updating;
- dimension reduction as representation learning.

Today

We introduce **neural networks**: flexible functions built by composing simple differentiable layers, trained by gradient-based loss minimisation.

Lecture 10 will focus on neural networks for images, especially convolutional neural networks.

Neural networks in one sentence

Neural network

A neural network is a parameterised function

$$f_{\theta} : \mathbb{R}^p \longrightarrow \mathbb{R}^q$$

constructed by composing simple layers, then trained by minimising a loss function over data.

Same idea as before:

- make predictions $\hat{y} = f_{\theta}(x)$;
- measure loss $L(y, \hat{y})$;
- update parameters θ .

New ingredient:

- f_{θ} is much more flexible;
- intermediate layers learn representations;
- gradients are computed automatically.

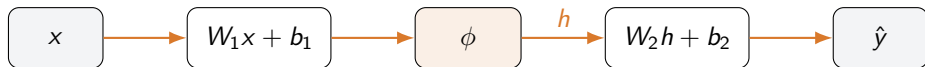
From linear models to neural networks

A linear model uses one affine map:

$$\hat{y} = w^\top x + b.$$

A neural network repeats the pattern:

affine map $\longrightarrow$ nonlinear activation ϕ .



The network is still just a function. The challenge is to train it reliably.

A representation-learning view

A one-hidden-layer network can be written as

$$h = \phi(W_1x + b_1), \quad \hat{y} = W_2h + b_2.$$

Interpretation

The hidden layer h is a learned representation of the input x .

PCA from Lecture 8

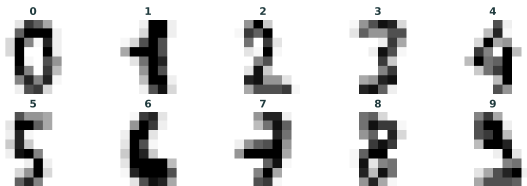
- unsupervised;
- linear projection;
- preserves variance/reconstruction.

Neural networks

- often supervised;
- nonlinear transformation;
- shaped by the prediction loss.

Running example: handwritten digits

Running example: handwritten digits as 64 input features



- Lecture 8 used PCA to visualise and compress these images.
- We can also train a network to predict the digit label.

Input

Each image is 8×8 , so each observation has $p = 64$ pixel features.

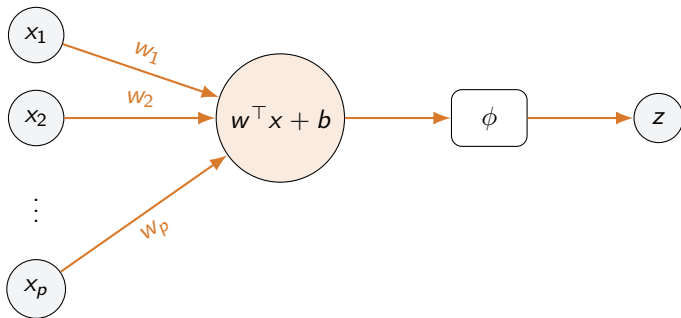
Building blocks

A neuron: affine score plus activation

For input $x \in \mathbb{R}^p$, a single neuron computes

$$a = w^\top x + b, \quad z = \phi(a),$$

where $w \in \mathbb{R}^p$ is a weight vector, $b \in \mathbb{R}$ is a bias and ϕ is an activation function.



From neurons to layers

A layer computes many neurons at once:

$$h = \phi(Wx + b),$$

where

$$W \in \mathbb{R}^{m \times p}, \quad b \in \mathbb{R}^m, \quad h \in \mathbb{R}^m.$$

Shape intuition

If x has p features and the layer has m hidden units, then W has mp weights and b has m biases.

Stacking layers gives a multilayer perceptron (MLP):

$$h^{(0)} = x, \quad h^{(\ell)} = \phi(W_\ell h^{(\ell-1)} + b_\ell).$$

Why nonlinear activations are essential

Suppose there are no activation functions. Then two layers collapse into one linear model:

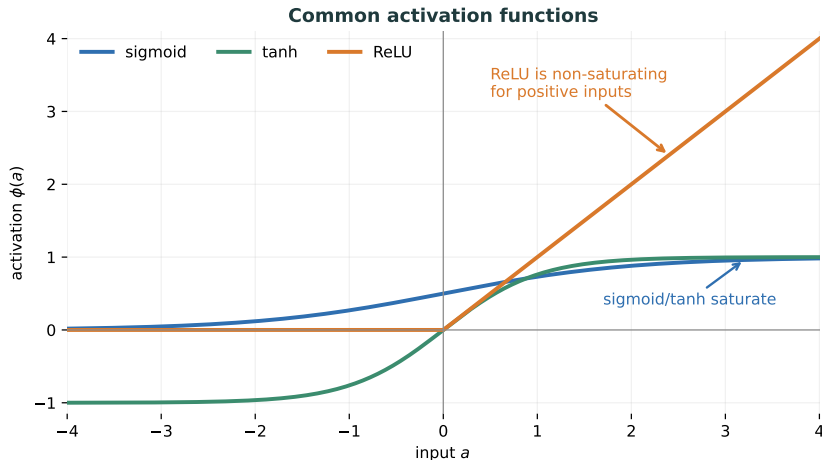
$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

Key point

Without nonlinear activations, a deep neural network is still just a linear model.

Activation functions let networks form nonlinear decision boundaries and nonlinear representations.

Common activation functions

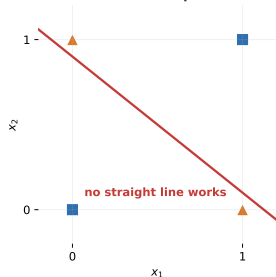


- Sigmoid and tanh are smooth but can saturate, giving very small gradients.
- ReLU, $\text{ReLU}(a) = \max(a, 0)$, is simple, widely used and often avoids saturation for positive inputs.

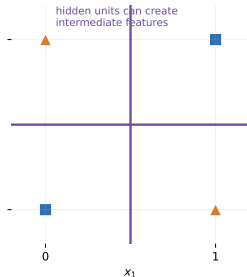
A small example: XOR

The XOR function returns 1 when exactly one of two binary inputs is 1.

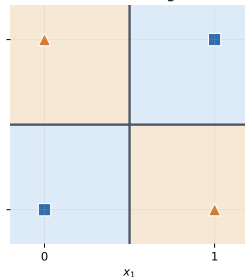
One line cannot separate XOR



Two hidden features



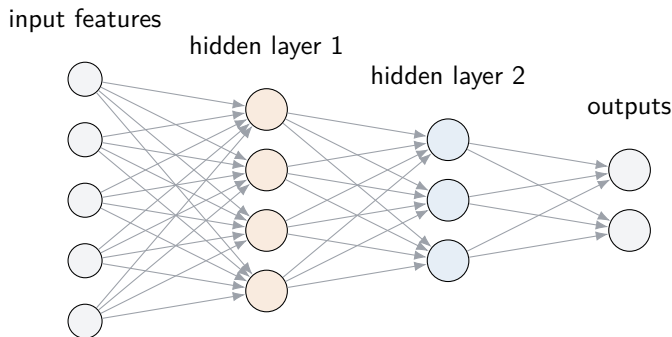
MLP decision regions



Lesson

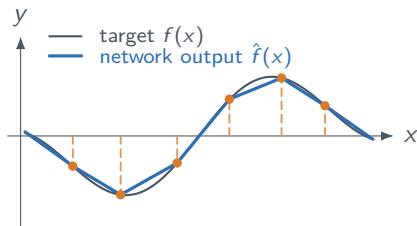
A single linear separator cannot solve XOR. A hidden layer can create intermediate features that make the problem linearly separable at the output layer.

A multilayer perceptron



Depth is often counted as the number of trainable affine layers; this example has **two hidden** layers and a **depth** of 3 (includes output layer). Width means the number of units in a hidden layer. The architecture controls the family of functions we can learn.

Why hidden layers are expressive



Informal universal approximation

On a bounded input range, there exists a neural network with enough nonlinear hidden units that can approximate any continuous target function arbitrarily well.

A one-dimensional ReLU network is a piecewise linear function: each hidden unit adds one kink, so more units give a finer approximation.

Important caveat

This says suitable weights *exist*; it does not guarantee easy training, enough data, or good validation performance.

Counting parameters

Consider the small digit classifier used in the companion notebook:

$$64 \longrightarrow 32 \longrightarrow 10.$$

First affine layer

$$64 \times 32 \text{ weights} + 32 \text{ biases} = 2080.$$

Output affine layer

$$32 \times 10 \text{ weights} + 10 \text{ biases} = 330.$$

$$\text{Total parameters: } 2080 + 330 = 2410.$$

Why count?

Parameter count gives a rough sense of model capacity. For large images, fully connected layers become expensive. This motivates CNNs (more on this later in Lecture 10).

Outputs and losses

The last layer should match the prediction task.

Task	Typical network output	Typical loss
Regression	real number $\hat{y} \in \mathbb{R}$	squared error $(y - \hat{y})^2$
Binary classification	logit $s \in \mathbb{R}$ or probability p	binary cross-entropy
Multiclass classification	logits $s \in \mathbb{R}^K$	cross-entropy

Terminology

Logits are raw class scores before applying a sigmoid or softmax.

Our main running example, classifying handwritten digits, is an example of multiclass classification.

Softmax and cross-entropy

For K classes, the network produces logits

$$\mathbf{s} = (s_1, \dots, s_K) \in \mathbb{R}^K.$$

The softmax converts logits into probabilities:

$$p_k = \frac{\exp(s_k)}{\sum_{j=1}^K \exp(s_j)}.$$

If the true class is c , the cross-entropy loss is

$$L = -\log p_c.$$

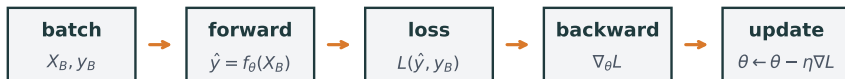
Interpretation

The loss is small when the model assigns high probability to the true class.

Training neural networks

The training loop

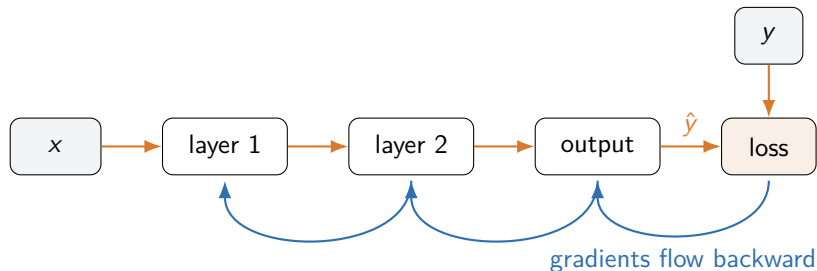
The same five steps appear in almost every PyTorch training loop



- 1 Choose a mini-batch of examples.
- 2 Run the network forward to compute predictions.
- 3 Compute the loss.
- 4 Backpropagate to compute gradients.
- 5 Update the parameters with an optimiser.

An **epoch** is one complete pass through the training set.

Forward pass, loss, backward pass



Backpropagation

Backpropagation is efficient bookkeeping for the chain rule. It computes how changing each parameter would change the loss.

Backpropagation in one-dimensional notation

Suppose a simple network computes

$$a = w_1x + b_1, \quad h = \phi(a), \quad \hat{y} = w_2h + b_2,$$

and loss $L(y, \hat{y})$.

To update w_1 , the chain rule gives

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial w_1}.$$

Practical message

You do not compute these derivatives by hand in PyTorch. Autograd builds the computational graph and applies backpropagation automatically.

Mini-batch gradient descent

For training data $(x^{(i)}, y^{(i)})_{i=1}^N$, the empirical risk is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f_{\theta}(x^{(i)})).$$

Full-batch gradient descent uses all examples:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).$$

Mini-batch gradient descent approximates this using a small batch B_t :

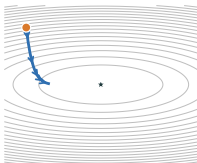
$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \ell(y^{(i)}, f_{\theta_t}(x^{(i)})).$$

This is the workhorse behind modern neural-network training.

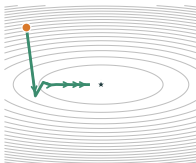
The learning rate matters

The learning rate controls the size of each update

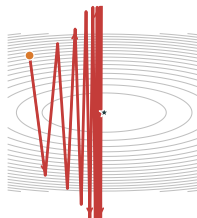
$\eta = 0.025$: **too small**



$\eta = 0.12$: **about right**



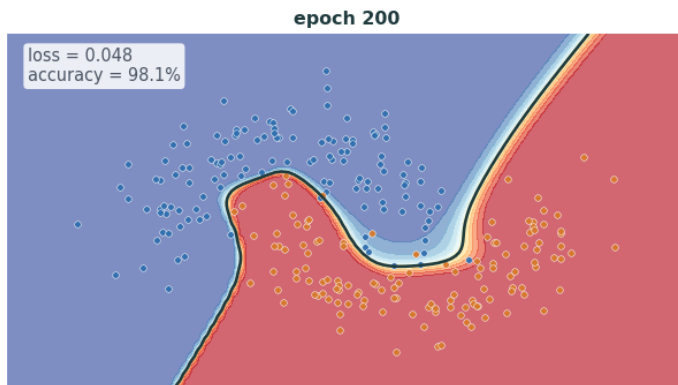
$\eta = 0.21$: **too large**



- Too small: learning is slow.
- About right: steady progress.
- Too large: updates can oscillate or diverge.

Training a nonlinear decision boundary (animated)

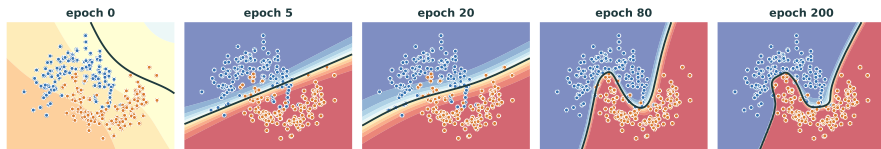
A neural network learning a nonlinear decision boundary



The loss shapes the decision boundary as the parameters are updated.

Static view of training dynamics

Training gradually bends the decision boundary



Caution

Training a network is optimisation in a high-dimensional, usually non-convex parameter space. Good practice matters.

What does an optimiser do?

Gradient descent is the basic idea:

$$\theta_{t+1} = \theta_t - \eta g_t.$$

In practice, common optimisers modify this update:

- **SGD with momentum:** averages recent gradients to smooth updates;
- **Adam:** adapts step sizes separately for different parameters.

A common add-on is **weight decay** which shrinks weights at each update to reduce overfitting.

For this lecture

The essential idea is still: compute gradients, then update parameters in a direction that reduces the loss.

Generalisation & practice

Model capacity and overfitting

Neural networks can have many parameters, so the same principles from earlier lectures still apply.

Too little capacity

- high training loss;
- model underfits;
- representation too simple.

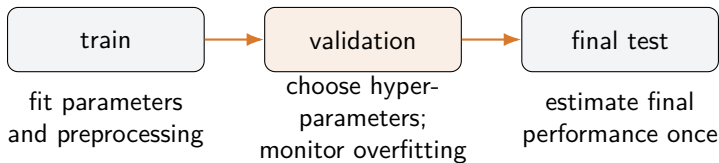
Too much capacity

- very low training loss;
- poor validation/test performance;
- model memorises noise.

The goal is not to minimise training loss at all costs. The goal is good generalisation.

Train, validation and test sets

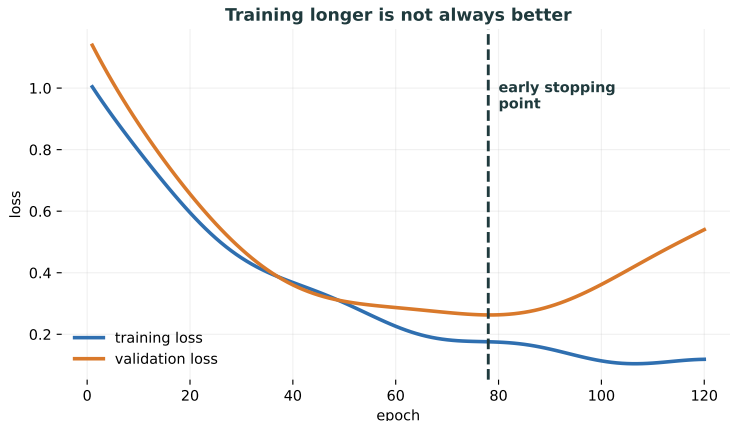
For neural networks, it is tempting to watch every metric all the time. Resist that temptation.



Good practice

Use the validation set during model development. Keep the test set untouched until the final evaluation.

Training and validation curves



How to read this plot

Monitor validation loss during model development. Keep the test set for the final evaluation after architecture and learning-rate choices are fixed.

When training fails: common symptoms

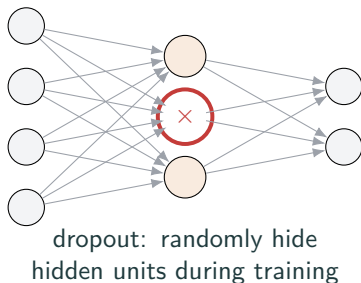
Symptom	Possible explanation
Loss barely decreases	learning rate too small; model too simple; inputs not scaled
Loss is unstable	learning rate too large; bad preprocessing; numerical issue
Training accuracy high, validation accuracy low	overfitting; train/validation mismatch
Validation/test score looks suspiciously good	data leakage; test-set reuse during tuning
Predictions mostly one class	class imbalance; wrong loss/output shape; optimisation problem

Debugging habit

Before training for many epochs, check tensor shapes and try to overfit a tiny batch. If the model cannot fit a few examples, something is probably wrong.

Common tools

- **early stopping**: stop when validation loss stops improving;
- **weight decay**: penalise large weights;
- **dropout**: randomly hide hidden units during training;
- **data augmentation**: create realistic transformed inputs.



Practical checklist before training

- **Split data first:** fit preprocessing only on the training set.
- **Scale inputs:** neural networks usually train better with standardised inputs.
- **Use validation data:** monitor validation loss and accuracy; do not tune on the test set.
- **Check shapes:** most neural-network bugs are tensor-shape bugs.
- **Start simple:** small model, few epochs, overfit a tiny batch as a sanity check.

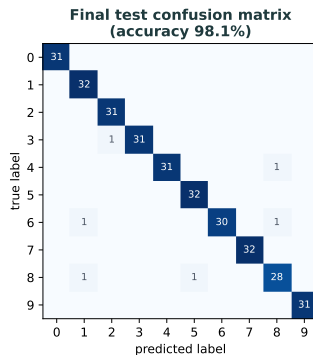
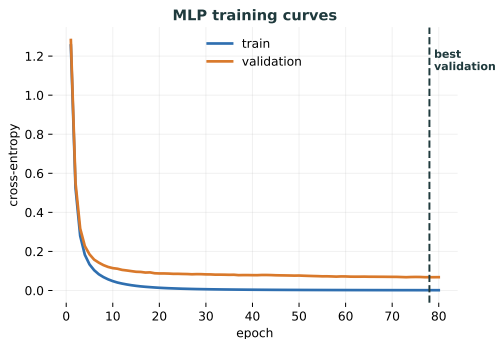
Reminder from earlier lectures

Preprocessing learned from test data is data leakage.

Looking ahead to the problem class

MLP on handwritten digits

This experiment will appear in the problem class.



Training curves use the validation set during development; the confusion matrix is for the final test set. Across the visualised number of epochs, the validation loss has essentially plateaued!

Diagnosing predictions

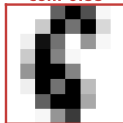
true 8 → pred 1
conf 0.99



true 6 → pred 1
conf 0.93



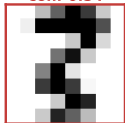
true 6 → pred 8
conf 0.88



true 4 → pred 8
conf 0.81



true 3 → pred 2
conf 0.54



true 8 → pred 5
conf 0.41



true = pred 0
conf 0.52



true = pred 0
conf 0.55



red frame = mistake; orange frame = lowest-confidence correct

Good habit

After plotting aggregate metrics, inspect individual examples. Mistakes often reveal data issues or model limitations.

The PyTorch training loop

Almost every training loop has this structure

```
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    logits = model(X_batch)
    loss = loss_fn(logits, y_batch)
    loss.backward()
    optimizer.step()
```

- `model(X_batch)` computes the forward pass.
- `loss.backward()` computes gradients by backpropagation.
- `optimizer.step()` updates the parameters.

Suggested activity

Go through the companion notebook and play around with the code; e.g., alter the number of hidden units or the learning rate and explore how things change.

Mini-exercise: before touching code

Suppose we train an MLP on digit images.

Discuss with a neighbour:

- ① What is the input dimension if each image is 8×8 ?
- ② How many output logits should the network produce for ten digit classes?
- ③ Which loss should we use for multiclass classification?
- ④ What would it mean if training accuracy is high but validation accuracy is low?

Answer sketch

Input dimension 64; output dimension 10; cross-entropy loss; likely overfitting or train/validation mismatch.

Summary

- Neural networks are flexible functions built by composing layers.
- A layer computes $h = \phi(Wx + b)$.
- Nonlinear activations are essential: otherwise deep networks collapse to linear models.
- Outputs and losses must match the task.
- Parameter counts give a concrete sense of model capacity.
- Backpropagation computes gradients efficiently using the chain rule.
- Mini-batch gradient descent trains networks at scale.
- Validation curves, test sets, regularisation and careful preprocessing are essential for generalisation.